

CASO DE ESTUDIO · DEVOPS EN LA PRÁCTICA

ESDOMED Services

Cómo un portal hospitalario real combina contenedores, infraestructura como código y despliegue continuo.



Docker



Docker Compose



Firebase

▲ Vercel



Git / GitHub



IaC



¿Qué es DevOps?

No es una herramienta ni un puesto: es una **cultura + prácticas + herramientas** que unen Desarrollo (Dev) y Operaciones (Ops) para entregar software rápido, seguro y repetible.



Cultura

Colaboración entre quien programa y quien opera. Responsabilidad compartida del ciclo completo.



Automatización

Construir, probar, desplegar y configurar sin pasos manuales: menos error humano, más velocidad.



Mejora continua

Ciclos cortos, feedback constante y entornos reproducibles de desarrollo a producción.

01

Plan

Git Issues / tablero

02

Code

TypeScript · ESLint

03

Build

Docker · npm build

04

Release

Vercel

05

Deploy

Vercel · Firebase

06

Operate

Firebase BaaS

¿Qué es ESDOMED Services?

Portal operativo interno del servicio de Estadística y Documentos Médicos de un hospital. Conecta a médicos y personal administrativo en flujos de trabajo clínicos.



Traslados

Solicitud y confirmación de traslados de servicio o cama.



Fallecidos

Notificación y registro de pacientes fallecidos.



Impresiones

Carga de PDFs y control de impresión de documentos.



Expedientes

Gestión de pacientes, ingresos, altas e incapacidades.

- ▶ **6 roles** distintos: médico, esdomed, admin, enfermería, psicología, trabajo social.
- ▶ **Acceso por rol** validado en el frontend y en reglas de servidor.
- ▶ Datos sensibles de salud → la **seguridad como código** es central.

```
STACK PRINCIPAL Next.js 16 · App Router // framework full-  
stack React 19 + TypeScript 5 Tailwind CSS 4 Firebase // Auth  
· Firestore · Storage Docker + Compose // entorno local  
Vercel // hosting + deploy continuo
```

PARTE 02

Las herramientas DevOps del proyecto, una a una

Control de versiones · Contenedores · Orquestación · Infraestructura como código · Paridad de entornos · Cloud · Despliegue continuo · Calidad · Secretos.

Git & GitHub

La base de todo flujo DevOps: historia del código, colaboración y el disparador del despliegue automático.

- ▶ **Git** versiona cada cambio; ramas para trabajar en paralelo.
- ▶ **GitHub** aloja el repositorio remoto y centraliza la colaboración.
- ▶ Cada push a la rama principal **dispara un despliegue** en Vercel (la "C" de CI/CD).
- ▶ `.gitignore` protege secretos y artefactos: `service-account.json`, `.env*`, `node_modules`.

Por qué importa: sin control de versiones no hay automatización confiable. Git es el "contrato" que el resto de las herramientas observan.

```
.GITIGNORE - EXTRACTO # env files .env* # Firebase service
account - NUNCA commitear service-account.json *-service-
account.json # vercel / next / typescript .vercel /.next/
*.tsbuildinfo
```

Docker

Empaqueta la app y sus dependencias en imágenes reproducibles. "Funciona en mi máquina" deja de ser un problema: funciona igual en cualquier máquina.

- ▶ Imagen ligera basada en `node:20-alpine`.
- ▶ Una imagen aparte para los **emuladores de Firebase** (requieren Java → `eclipse-temurin`).
- ▶ `.dockerignore` evita copiar `node_modules`, `.next` o secretos a la imagen.
- ▶ Entorno idéntico para todo el equipo, sin instalar nada local salvo Docker.

Patrón clave: multi-stage build — copia el JRE de una imagen Java a la imagen Node solo para correr los emuladores. Imagen más pequeña y enfocada.

```
DOCKERFILE.DEV FROM node:20-alpine WORKDIR /app COPY
package*.json ./ RUN npm install EXPOSE 3000 CMD ["npm",
"run", "dev:docker"] DOCKERFILE.EMULATORS (MULTI-STAGE) FROM
eclipse-temurin:21-jre AS jre FROM node:20-slim COPY --
from=jre $JAVA_HOME $JAVA_HOME
```

Docker Compose

Levanta todo el entorno de desarrollo con **un solo comando**: la app web y los emuladores de Firebase, conectados y coordinados entre sí.

- ▶ **2 servicios**: app (Next.js) + emulators (Firebase).
- ▶ **healthcheck**: comprueba que los emuladores estén vivos antes de arrancar la app.
- ▶ **depends_on**: **condition: service_healthy** → **orden de arranque garantizado**.
- ▶ **volumes** montan el código → **hot reload**, y cachean los binarios de emuladores.
- ▶ Variables de entorno inyectadas por servicio.

Un comando: `docker compose up` reproduce el entorno completo en cualquier equipo, sin pasos manuales.

```
DOCKER-COMPOSE.YML - EXTRACTO services: emulators: build: {  
  dockerfile: Dockerfile.emulators } ports: ["4000:4000",  
  "8080:8080"] healthcheck: test: ["CMD", "curl", "-  
  f", "localhost:4000"] app: ports: ["3000:3000"] depends_on:  
  emulators: { condition: service_healthy }
```

Seguridad como código (IaC)

Las reglas de acceso a la base de datos y al almacenamiento **viven en el repositorio**, versionadas junto al código. Cambiar permisos = un commit revisable, no un clic en una consola web.

- ▶ `firestore.rules` — quién lee/escibe cada colección según su rol.
- ▶ `storage.rules` — acceso a archivos subidos.
- ▶ `firebase.json` — configuración declarativa de emuladores y reglas.
- ▶ Funciones reutilizables: `isEsdomed()`, `isMedico()`, `isAdmin()`...

Ventaja DevOps: la infraestructura es **auditable y reproducible**. El historial de Git muestra quién cambió qué permiso y cuándo.

```
FIRESTORE.RULES - EXTRACTO function getRole() { return
get(/databases/${db}/documents/ usuarios/${uid()}).data.role;
} function isEsdomed() { return isAuth() && getRole() =
'esdomed'; } match /traslados/{id} { allow create: if
isMedico(); allow update: if isStaff(); }
```


Emuladores de Firebase

Una copia local y completa de la nube — Auth, Firestore y Storage — corriendo en contenedor. Se desarrolla y prueba **sin tocar datos reales de producción**.



Auth · 9099

Login y usuarios de prueba sin afectar cuentas reales.



Firestore · 8080

Base de datos local con las mismas reglas que producción.



Storage · 9199

Subida de PDFs simulada localmente.

- ▶ **Dev/prod parity** — principio clave de DevOps (12-Factor App).
- ▶ El código detecta `NEXT_PUBLIC_USE_EMULATORS` y se conecta a local automáticamente.
- ▶ UI de emuladores en `:4000` para inspeccionar datos.

```
SRC/LIB/FIREBASE.TS if (process.env.NEXT_PUBLIC_USE_EMULATORS
=== "true") { connectAuthEmulator(auth,
"http://localhost:9099"); connectFirestoreEmulator(db,
"localhost", 8080); connectStorageEmulator(storage,
"localhost", 9199); }
```

Firebase (BaaS)

La nube de Google que provee backend listo para usar: autenticación, base de datos en tiempo real y almacenamiento — sin administrar servidores (modelo serverless).



Auth

Authentication

Login por email/contraseña; el rol se carga desde Firestore al iniciar sesión.



DB

Firestore

Base NoSQL en tiempo real: traslados, fallecidos, pacientes, usuarios.



Files

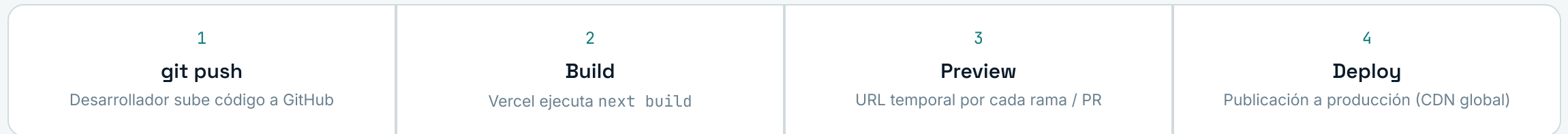
Storage

Almacenamiento de los PDFs que suben los médicos.

- ▶ **Serverless / Ops gestionadas:** Google opera, escala y respalda. El equipo no administra infraestructura.
- ▶ **Firestore Admin SDK** en rutas API para operaciones privilegiadas (gestión de usuarios).
- ▶ Misma configuración para emulador y nube → **cambiar de entorno es una variable.**
- ▶ Las reglas laC protegen los datos en ambos.

Vercel — CI/CD & Hosting

Plataforma conectada al repositorio de GitHub. Cada cambio en el código se construye y se publica **automáticamente**, sin intervención manual.



- ▶ **Despliegue continuo (CD):** de push a producción sin pasos manuales.
- ▶ **Preview deployments:** cada rama obtiene su propia URL para revisar antes de fusionar.
- ▶ **Rollback** instantáneo a cualquier despliegue anterior.

```
.VERCEL/PROJECT.JSON { "projectId": "prj_R4Ae...teiro4A",  
  "orgId": "team_Qffao...2p1Mns", "projectName": "esdomed-  
services" } // vincula el repo con el proyecto Vercel
```

Quality gates & "shift-left"

Detectar errores lo antes posible — al escribir código, no en producción — y mantener la configuración fuera del código.



TypeScript

Tipado estático: errores detectados al compilar, antes de desplegar. Bloquea el build si algo no encaja.



ESLint

Análisis estático de estilo y errores comunes. Estándar de calidad consistente para todo el equipo.



Gestión de secretos

Variables en `.env.local` y credenciales fuera de Git. Config separada del código (12-Factor).

- ▶ **Automatización de tareas** — scripts versionados de import y limpieza de datos.
- ▶ Modo `--dry-run`: simula antes de ejecutar cambios reales.

```
PACKAGE.JSON — SCRIPTS "build": "next build", "lint":  
"eslint", "importar:dry": "node ../importar.mjs --dry-run",  
"importar": "node ../importar.mjs"
```

Cada herramienta, en su fase DevOps

FASE DEL CICLO	HERRAMIENTA EN EL PROYECTO	QUÉ RESUELVE
Plan / Code	Git · GitHub · TypeScript · ESLint	Versionado, colaboración y calidad antes de desplegar
Build	Docker · Docker Compose · npm	Empaquetado reproducible y entorno local de un comando
Test (local)	Emuladores Firebase	Probar contra una nube local, sin datos reales
Release / Deploy	Vercel (conectado a GitHub)	Despliegue continuo automático + previews + rollback
Operate	Firebase (Auth · Firestore · Storage)	Backend serverless gestionado y escalable
Secure (transversal)	firestore.rules · storage.rules (IaC)	Seguridad versionada y auditable como código

Lectura: el proyecto cubre de forma real las fases **Code** → **Build** → **Test** → **Release** → **Deploy** → **Operate**, con la seguridad tratada como código de forma transversal.

¿Cumple como caso DevOps?

Sí — aplica varios pilares reales de DevOps con herramientas modernas. Pero un buen análisis también reconoce lo que falta.

✓ Lo que Sí implementa

- ▶ Contenedores y orquestación local (Docker + Compose)
- ▶ Infraestructura/seguridad como código (rules versionadas)
- ▶ Despliegue continuo automático (Vercel ← GitHub)
- ▶ Paridad dev/prod con emuladores
- ▶ Quality gates: TypeScript + ESLint
- ▶ Gestión de secretos y configuración

X Lo que aún falta (próximos pasos)

- ▶ Pipeline de CI explícito (GitHub Actions)
- ▶ Pruebas automatizadas (unitarias / e2e)
- ▶ Monitorización y observabilidad (logs, alertas, métricas)
- ▶ IaC de infraestructura (Terraform / contenedores en prod)
- ▶ Gestión de secretos centralizada (vault)

Conclusión: es un **caso de estudio válido y honesto** de DevOps aplicado a un proyecto real de tamaño mediano. No es una plataforma DevOps "de manual completo", y precisamente eso lo hace ideal para discutir **qué herramientas hay y qué seguiría faltando**.

PARA LLEVARSE DE LA CLASE

3 ideas clave



01

DevOps es flujo

El valor no está en una herramienta suelta, sino en cómo Git, Docker, IaC y Vercel se encadenan sin pasos manuales.

02

Reproducibilidad

Contenedores + emuladores + reglas como código = el mismo entorno para todos, de local a producción.

03

Automatización

De git push a producción en minutos, con calidad verificada antes de desplegar.

¿Preguntas? — Gracias.